

6COSC023W

Computer Science Final Project

Final Year Project (FYP) - Report

The Shape of Sound: A Practice-Led Study in Creative Human- Computer Interaction

Student: Oliver Bryan (w1980003)

Supervisor: Andrea Martina

Degree: BEng Software Engineering

School of Computer Science & Engineering

University of Westminster

2026

Declaration

This report has been prepared based on my own work. Where other published and unpublished source materials have been used, these have been acknowledged in references.

Word Count: TODO

Student Name: Oliver Bryan

Date of Submission: 24/04/26

Abstract

The Shape of Sound is a practice-led project that investigates whether a synesthetic, visual-first interface can make sound synthesis more intuitive without reducing creative depth.

The project aims to address a common split in music software: powerful professional tools often require specialist technical knowledge, while beginner-friendly tools are accessible but offer limited control over how sound is shaped. In response, this project offers a web-based application where users manipulate visual properties and receive immediate sonic feedback, turning synthesis interaction into a more exploratory and engaging process.

The study combines iterative design practice, implementation, and reflective analysis. Early development included a 2D Konva JS based interface submitted at IPD stage. Some reflection led to a re-exploration in 3D to better support spatial interaction and expressive shape manipulation. The final implementation uses Next.js 15 for interface architecture, Three.js with React Three Fiber for interactive graphics, and Tone.js for browser-based synthesis. Core mappings connect shape preset, colour/note, octave, size, roundness, wobble, and envelope parameters to audible synthesis behaviour, with real-time keyboard triggering and export support.

The final artefact is a complete web application that demonstrates a coherent synesthetic workflow for creative sound design, combining real-time visual interaction, synthesis control, and export functionality. Evaluation results show that participants were able to create and modify sounds quickly, and that the visual mapping reduced perceived complexity when compared with parameter-first interaction. The project offers a grounded approach to translating shape interaction into sound design control, and reflects on how iterative prototyping can balance accessibility with creative depth. Future work is focused on extending the feature set, with sequencer functionality intentionally positioned as post-submission development.

Current in-progress items (to complete and update later):

- **In progress:** Full product completion and integration polish across interaction, audio, and export flows.
- **In progress:** Formal user evaluation design, participant sessions, and results analysis.
- **In progress:** End-to-end testing, bug fixing, and performance/usability validation.
- **In progress:** Final reflective analysis linking outcomes to research questions.
- **In progress:** Future roadmap definition with sequencer explicitly positioned as post-submission work.
- **In progress:** Replace generic results language with concrete evaluation findings and key metrics.

Acknowledgements

I would like to thank my supervisor, Andrea Martina, for supporting the direction of this project, assisting with scope definition, and providing valuable feedback throughout the prototype stages.

I would also like to thank the participants of the user testing sessions for their time and insightful feedback, which was crucial in improving the usability and functionality of the application.

Finally, I would like to acknowledge the open-source communities building the tools and libraries used in this project, including React, Three.js, and Tone.js, whose work made this project possible.

Contents

Declaration	2
Abstract	3
Acknowledgements	4
List of figures	6
List of tables	7
1 Introduction	8
1.1 Problem Statement	8
1.2 Aims and objectives	10
2 Background	10
2.1 Literature Survey	11
2.2 Review of projects	12
2.3 Review of tools, frameworks and techniques	14
3 Legal, Social, Sustainability, and ethical issues	16
4 Methodology	16
5 Design	18
6 Tools and Implementation	21
6.1 Tools used and justification	21
6.2 Implementation	23
7 Testing	27
7.1 Functional testing	27
7.2 User testing	30
8 Conclusions and Reflections	32
Bibliography	32

List of figures

Figure 1 Top self-reported music production struggles from an EDMProd survey of
1000+ producers [1]. 9

Figure 2 Results of my beginner survey on music production, “Have you ever felt that
music production software seems too complicated or intimidating”. 9

Figure 3 High-level project timeline and milestones. 16

List of tables

Table 1 Core interaction mappings used in The Shape of Sound. 19

Table 2 Functional test plan and outcomes for the final application. 29

Table 3 Summary of user testing questionnaire ratings. 30

Table 4 Summary of key feedback from user testing. 31

1 Introduction

The Shape of Sound is a practice-led project exploring a visual approach to audio synthesis through interactive shape manipulation. The project addresses two linked issues in music software: accessibility barriers for beginners and limited creative flexibility in many simplified tools. The primary purpose is education, the project is designed to help new users understand synthesis through direct visual interaction and immediate feedback, rather than being overwhelmed with technical jargon. The secondary purpose is creative support for experienced users by offering an alternative workflow for rapid idea generation. The project is positioned as HCI work, linking interface design choices to creative and learning outcomes.

1.1 Problem Statement

Studies of contemporary music artists identify “musician’s block” as a common phenomenon [2], often linked to music-making anxiety and challenges within the creative process. As shown in Figure 1, the top self-reported struggle is not knowing “where to start”. This is evermore common for beginners, who often find themselves overwhelmed by the technical complexity of music software, with a steep learning curve that requires understanding abstract concepts before engaging in creative expression. This creates friction and fatigue, leading to disengagement and frustration. Even experienced producers can get stuck in creative blocks when they use the same environment repeatedly [3], leading to stagnation and reduced motivation. A new way to make sounds may help a producer express themselves creatively and get out of a block.

From my own research, I found that ~66% of beginners surveyed felt that music production software seems too complicated or intimidating (see Figure 2). This is a significant barrier to entry, and it highlights the need for more accessible tools that can help users learn and create without being overwhelmed by technical details. It is often discussed that it takes around 10,000 hours of deliberate practice to achieve mastery in a field [4], but this can be discouraging for beginners who may not have the time or resources to invest in such a long learning process. These barriers can lead to a lack of diversity in music production, as only those with the resources and time to invest in learning complex software can participate fully. These barriers suggest a need for more intuitive, engaging, and accessible approaches to early-stage sound design that support both learning and sustained creative engagement.

Music Production Struggles

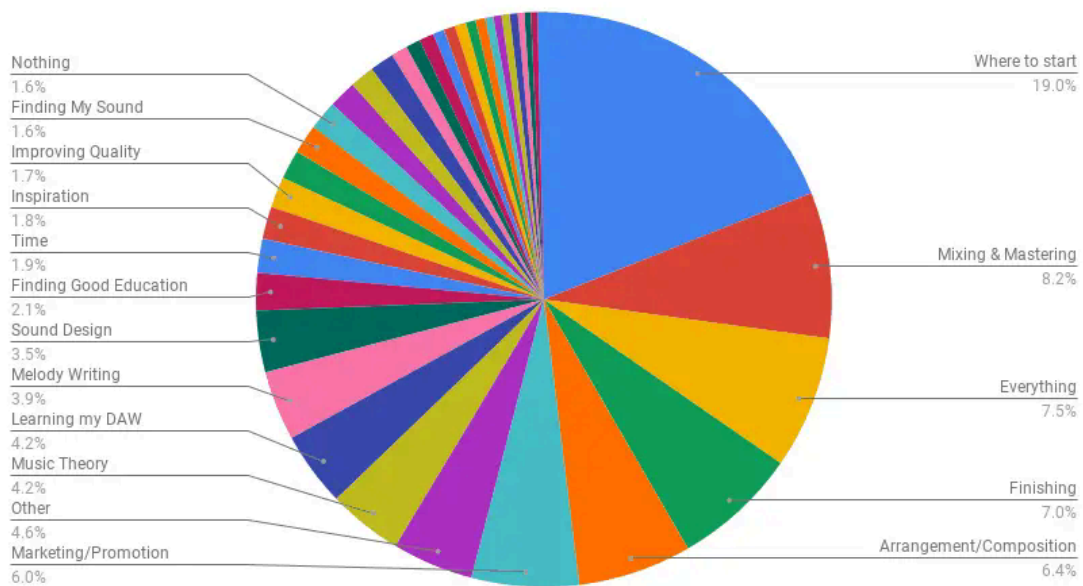


Figure 1: Top self-reported music production struggles from an EDMProd survey of 1000+ producers [1].

Have you ever felt that music production software (FL Studio, Ableton, Logic, etc.) seems too complicated or intimidating?

1

2

3

Not at all

☐

☐

☒

Yes, very

2 responses

1

2

3

Not at all

☐

☒

☐

Yes, very

1 response

Figure 2: Results of my beginner survey on music production, “Have you ever felt that music production software seems too complicated or intimidating”.

1.2 Aims and objectives

The Shape of Sound addresses the need for a more intuitive, engaging and accessible approach to early-stage sound design, by providing a simple starting point and a visual method of sound creation, with the aim of reducing barriers to entry while supporting creative expression for both beginners and experienced producers. The project aims to achieve this by developing a web-based application that allows users to manipulate visual properties of a shape and receive immediate sonic feedback, turning synthesis into a more exploratory and engaging process. The project also aims to assess the effectiveness of this approach through user testing and evaluation, with the goal of demonstrating that a synesthetic, visual-first interface can make sound synthesis more intuitive without reducing creative depth and technical possibilities.

A key objective is to investigate how visual properties of a shape can be meaningfully mapped to audio synthesis parameters, drawing on ideas from synesthesia, multisensory experience, and accessible music making. This will involve exploring the visual attributes of shapes (e.g. size, color, shape, movement) and how they can correspond to sound parameters (e.g. gain, pitch, waveform, modulation) in a way that is intuitive, interactive and engaging to users. The project will take form as a web application that allows users to manipulate visual properties of a shape and receive immediate sonic feedback, turning synthesis into an exploratory and novel process, hoping to prove that the visual-first model can reduce the perceived complexity of music production and synthesis as a whole. It should make core sound design concepts more approachable for beginners. The project will also explore how viable a visual-first approach to music production technology can be. This will be evaluated through user testing and reflective analysis, with the goal of demonstrating that a new interface can make audio synthesis more intuitive without harming the creative potential of the tool. Identifying the strengths, limitations, and future development opportunities of the approach will be a key contribution to practice-based HCI research, with the potential to influence innovation in music technology design and education.

2 Background

There are three main areas of background research that have informed the development of this project: relevant literature, comparable products, and the tools and techniques explored during development. The literature side covers multisensory interaction, synesthesia, sound symbolism, sonification, human-computer interaction, creativity, creative block, and barriers to learning music production. Together, these areas frame the project as both an interface-design problem and a creative-learning problem. The project review compares tools whose focuses range from accessibility to depth. FL Studio and Ableton are powerful professional environments with high control, but steeper learning curves. Incredibox and Splice Beatmaker represent more accessible visual and web-based experiences, but have much less direct synthesis control. Synesthesia.live is also relevant as a related audio-visual system, even though it maps in the opposite direction to this project. These comparisons help define the gap the project is aiming to fill: a visual-first, web-based synthesis tool that is approachable without

becoming toy-like. The technical review reflects the way the project evolved in practice. The development direction began around Next.js, Three.js, and Tone.js, with a move to 2D, with Konva.js. Later reflection led to a return to 3D for the final implementation. The technical review can show not just what tools were used, but why they were adopted.

2.1 Literature Survey

It has been found that learning often improves when information is presented through multiple sensory modalities rather than a single modality [5]. This supports the idea that pairing visual interaction with audio feedback can be more effective than relying solely on traditional synthesis parameters. Multimodal feedback can enhance learning and task understanding [6], which helps to frame the project's interface as a feedback-based learning tool rather than just a creative toy. The benefits of multisensory learning are particularly relevant to this project as it aims to support users in learning about sound design through a visual interface that provides immediate audio feedback, providing a multisensory learning experience. It is important that the project can provide a learning experience while remaining engaging and fun, which is supported by the research on multisensory learning benefits.

People also make consistent cross-sensory associations between audio qualities such as pitch and visual qualities like size and brightness [7], which supports designing perceptually meaningful mappings rather than arbitrary mappings. Sound symbolic associations can make links between sound and perceptual qualities feel intuitive even without true synesthesia [8], which is useful for explaining why users may understand the mappings without needing specialist knowledge or unusual sensory traits. The bouba/kiki effect supports this, showing a robust association between rounded or sharp shapes and different sound qualities across cultures [9], which is directly relevant to using shape and roundness as part of audio-visual interaction. The project's use of visual shapes to control timbre can be informed by this research, ensuring that the mappings feel intuitive and engaging for users, even those without a background in music production or sound design. By leveraging these cross-modal correspondences, the project can create a more intuitive and enjoyable user experience, making it easier for users to understand and manipulate sound through visual interaction. It also helps to justify the choice of using shape and visual qualities as a means of controlling sound, as these associations are supported by research and can enhance the user's ability to learn and create with the tool.

Scriabin's *clavier à lumières* provides an early historical example of mapping musical pitch to colour in performance and composition [10], which gives historical precedent for visual-musical mapping, even if this project extends the idea into interactive sound design. This acts as historical context for the project, showing that the idea of linking visual elements to musical parameters has been explored in various forms for over a century. While Scriabin's instrument focused on pitch-to-colour mapping, this project expands on the concept by exploring more complex timbre control through visual interaction, demonstrating how historical ideas can be reimagined and extended with

modern technology and design principles. While it expands the idea into a different domain, the project stays true to the pitch-colour mappings that Scriabin used.

Multisensory musical experiences can increase positive mood and reduce anxiety compared with audio-only experiences [11], which supports the idea that adding a strong visual layer may make music interaction more engaging and less intimidating. Sonification can be understood as sound generated in response to data and interaction rather than only as a final musical output [12], which helps justify treating visual changes in the interface as sound-generating input. Sonic interaction design proposes that sound is part of the interactivity of a system [13], which places the project within interaction design research as well as music production software. Together, these ideas position the project not simply as a music tool, but as an interactive system in which sound plays a crucial role in feedback, meaning, and engagement.

Beginner-focused musical tools need to balance low barriers to entry with expressive potential [14], which aligns strongly with the project's aim of supporting both new users and more experienced producers. Creativity support tools should aim for low thresholds, with high ceilings and wide walls [15], which gives a strong HCI basis for making the interface accessible without making it creatively shallow. Visual control of synthesis through natural cross-modal mappings already has research precedent [16], which shows that visual-first synthesis control is an existing research direction rather than a speculative idea. Unique and exploratory musical interfaces can reduce the complexity of synthesis parameter spaces by supporting interaction over raw parameter editing [17], which supports the move away from parameter-heavy workflows toward exploratory sound design. The emotional engagement focus of the project is vital, as emotion and expression are what drive the creative process. By creating a tool that is not only functional but also emotionally engaging, the project can help users feel more connected to their music and more motivated to explore and create. This is particularly important for beginner users who may feel intimidated by traditional music production software, as the visual layer can make the experience more approachable and enjoyable, encouraging them to experiment and learn without fear of making mistakes or feeling overwhelmed by complex interfaces. While it greatly supports new users, the emotional engagement aspect can also enhance the experience for more experienced producers, providing a fresh and inspiring way to interact with sound that can spark creativity and new ideas.

2.2 Review of projects

I have reviewed an array of music software and interactive audio-visual tools to understand the current landscape of products that relate to the project's goals. This review includes both professional digital audio workstations (DAWs) and more accessible, web-based music creation tools, as well as a related audio-visual system. The aim is to identify the strengths and weaknesses of these existing products, and to position the project within this landscape by identifying a gap that it can fill.

FL Studio is a powerful DAW that offers high creative depth, detailed sequencing, and mature production workflows. However, it still relies heavily on conventional DAW structures and technical literacy, which can be intimidating for beginners [18], [19]. This makes it a strong contrast case for the project, as it demonstrates that powerful music software can be conceptually dense and difficult for new users to navigate. Ableton Live is another professional DAW that combines clip-based creativity with professional production depth. Where FL Studio is especially associated with detailed sequencing and piano-roll control, Ableton Live is more strongly associated with clip-based experimentation and live-oriented workflows. While it is comparatively more experimental and workflow-oriented than FL Studio, it still operates through established DAW conventions rather than visual-first sound shaping [20]. This shows that even one of the more intuitive professional tools does not directly solve the problem of making synthesis visually engaging and educational. Both DAWs assume prior knowledge of synthesis and music production, only providing guidance on the software itself. The power is given to the user in the form of complex menus, routing, plugins, and parameter editing. This is great for giving the user control, but leaves them overwhelmed with controls and options. Ableton Live is also relevant because its documentation now foregrounds accessibility features such as screen reader support and high-contrast options [21]. This is useful for separating technical accessibility from conceptual accessibility, since a system can be more accessible in one sense while still being difficult for first-time users to understand.

Incredibox is an important precedent for visual-first music creation because it is highly approachable, playful, and immediate, allowing users to make satisfying musical results without needing production knowledge [22], [23]. Incredibox gives instant reward, and provides an option for low-risk experimentation, making it especially effective for new users who are interested in playing with music and exploring loop creation. However, the depth provided is limited, with a new user having no control over the underlying synthesis parameters, or even given a chance to understand them. This supports the idea that visual immediacy and educational value are possible, but the existing projects make a trade-off: they prioritize ease of use over the ability to deeply manipulate sound.

Splice Beatmaker occupies a similar space because it is browser-based, low-friction, and useful for fast musical ideation, but it is sample-centric rather than synthesis-centric [24], [25]. Splice Beatmaker being browser-based makes it accessible from any device with a web browser. Its value lies in showing how browser-based music tools can support fast idea generation and transition into wider production workflows, even if the creative model is sample-driven. The success of Splice Beatmaker helps position The Shape of Sound as more focused on original sound creation than on loop assembly from pre-made material.

Synesthesia.live is relevant because it treats visual interaction as a serious creative practice and supports advanced audio-reactive visual systems, even though it mainly maps sound into visuals rather than visuals into sound [26], [27]. This is important because it demonstrates that visuality in music tools does not have to be superficial. Visual systems can become expressive instruments in their own right, capable of

supporting high level creative work. However, Synesthesia.live is ultimately focused on visual performance and audio-reactive output rather than helping users understand or shape sound itself. In this sense, it is not a direct solution to the problem addressed by this project. Its value as a comparison lies in showing that visual interaction can carry real artistic seriousness, while The Shape of Sound inverts the more common audiovisual relationship by using visual interaction as the means of creating and controlling sound.

Together, these products reveal a gap between complex professional tools that offer depth but require significant production literacy, and highly accessible visual tools that remain limited in direct sound design control [18], [20], [22], [24]. This gap is where The Shape of Sound is positioned, as a visual-first and web-based synthesis tool that aims to be approachable without becoming toy-like. The comparisons between the types of existing tools justify the need for a tool that can combine the immediacy of visual interaction with the depth of synthesis control, while also being appealing to users with existing music production experience, not just new users. By filling this gap, the project can provide a unique and valuable tool for both learning and creative expression in music production.

2.3 Review of tools, frameworks and techniques

This section reviews the tools, frameworks, and implementation techniques that shaped the development of The Shape of Sound, with a focus on how each choice supported the project's visual, technical, and creative goals. The review is structured around the main technologies used, with discussion of their advantages, limitations, and relevance to the project's aims.

Next.js 15 is most relevant as application infrastructure, giving the project a structured web framework for interface, routing, persistence, and deployment rather than acting as the creative engine itself [28], [29]. Next.js provides a solid foundation for building a web application, with features like server-side rendering, API routes, and built-in support for React. However, it is not directly responsible for the core audio-visual interaction design of the project. Its value lies in enabling a well-structured and performant web application that can support the creative features built on top of it. React 19 provides the component and state model for controls, views, and shared interaction logic, but it too does not solve graphics or audio on its own [30]. React was essential for organising parameter controls, views, and synchronised interface state, allowing the project to manage a complex interactive UI without losing the modularity needed for iterative development. TypeScript was also important to the project because it provided stronger type safety across UI state, shape parameters, and audio mappings, helping to make the system easier to maintain as it grew in complexity.

Three.js supports the visual-first interface, making 3D form part of the interaction rather than purely decorative presentation [31], [32]. Using Three.js introduced extra complexity, but the interactivity of the final result justifies its introduction. Konva.js was a valuable tool as part of the intermediate 2D phase. It supported rapid prototyping,

without needing to deal with the added complexity of 3D graphics in the browser [33]. Although the final implementation returned to 3D, Konva proved valuable as an intermediate prototyping stage for simplifying and testing interaction ideas. Its limitation was that it could not support the same sense of spatial richness and sculptural interaction as the final 3D approach.

Tone.js is a Web Audio API wrapper library that provides higher-level abstractions for timing, synths, and effects, making it easier to create complex audio interactions without needing to manage low-level audio nodes directly [34], [35]. This allowed the project to focus on mapping and interaction design rather than building an audio engine from scratch. However, it is important to note that Tone.js is built on top of the Web Audio API, so the underlying capabilities and constraints of the system still come from native browser audio infrastructure. The Web Audio API therefore remains the true technical foundation of the project's sound engine, even where the higher-level abstractions are used.

Tailwind CSS supported speed and consistency in building the surrounding interface, but it was secondary to the project's actual audiovisual contribution [36]. Tailwind primarily supported rapid interface styling, allowing development effort to remain focused on the audiovisual interaction model. It is a useful tool for building a polished interface, but it does not directly contribute to the core creative features of the project. Better Auth is relevant mainly where the project includes accounts and saved work, as it provides infrastructure for user authentication and session management [37]. Drizzle ORM in conjunction with Neon's PostgreSQL hosting supports the persistence of user data and saved projects in a flexible and modern way [38], [39]. These tools support user accounts and project persistence, but they remain infrastructure choices rather than central parts of the project's audiovisual interaction design. For exporting sounds, the project uses wasm-media-encoders, which provides a way to encode audio in the browser for export without needing server-side processing [40]. This is crucial for making the project practical for real creative workflows, as it allows users to export their creations directly from the browser without needing to record the sound themselves. The ability to export is a key part of the project's value proposition, as it allows users to take their creations beyond the live session and use them in other contexts. This makes wasm-media-encoders an important part of the technical stack, as it connects the interactive sound generation to portable output formats that can be used in wider production workflows.

While the previous tools support the wider application, the core behaviour of The Shape of Sound depends on a smaller set of implementation techniques that define how the tool actually works. Visual-to-audio parameter mapping is the most fundamental of these, as it determines how shape properties are connected to synthesis parameters and therefore how the instrument is experienced by the user [41], [42]. The design of these mappings is crucial, because users need to understand how visual actions affect sound in a way that feels intuitive, learnable, and creatively rewarding. Once these relationships are established, waveform blending and crossfading become important, because audio

feedback needs to match visual representation if the instrument is to avoid perceptual jumps [43], [44]. Modulation mapping, such as wobble or tremolo-style behaviour, extends this further by allowing visual movement to become audible behaviour rather than leaving shapes sonically static [45], [46]. In parallel, shape interpolation and morphing support gradual visual transformation, but these changes only become musically convincing when paired with matching sound transformation [47], [48]. Finally, browser-based export workflows are important because a web instrument becomes far more practical when users can record and keep their results rather than only hearing them in-session [49], [50]. Taken together, these techniques ensure that The Shape of Sound functions not simply as a visual novelty, but as a coherent instrument whose interaction model is expressive and useful within real creative workflows.

3 Legal, Social, Sustainability, and ethical issues

4 Methodology

This project followed an iterative prototyping methodology within a broader practice-led and agile development process. This approach was appropriate because the central challenges of The Shape of Sound were not only technical, but also interactive and experiential, requiring ideas to be tested through repeated design, implementation, and revision. Rather than treating development as a sequence of fixed steps, the project evolved through cycles of prototyping and reflection, allowing the visual-to-audio interaction model to be refined over time. At a higher level, the work still followed clear project stages, including research, requirements elicitation, interface design, core development, testing and refinement, and final delivery. The staged plan was useful for setting milestones, managing scope, and maintaining progress across the different phases of the project. However, these phases were not carried out as strictly isolated steps. Instead, each stage involved smaller cycles of experimentation, reflection, and revision, allowing the project to remain agile while still following a clear overall timeline.

Alongside this iterative approach, the project was also organised through a higher-level staged plan, represented in the Gantt chart shown in Figure 3. This structure divided the work into clear phases, including research, requirements elicitation, interface design, core development, testing and refinement, and final delivery. The staged plan was useful for setting milestones, managing scope, and maintaining progress across the different phases of the project. However, these phases were not carried out as strictly isolated steps. Instead, each stage involved smaller cycles of experimentation, reflection, and revision, allowing the project to remain agile while still following a clear overall timeline.

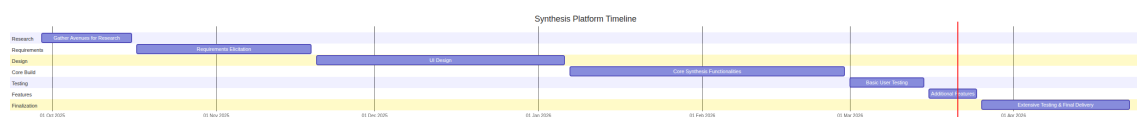


Figure 3: High-level project timeline and milestones.

This iterative process is reflected clearly in the design evolution of the project. The initial direction focused on a 3D implementation, as this best matched the ambition of creating a visually expressive interface for sound design. However, during development it became useful to simplify the interaction model through a 2D Konva-based stage, which made it easier to test core ideas such as shape manipulation, mapping behaviour,

and overall usability without the added complexity of 3D scene management. Once these interaction principles became clearer, the project returned to a final 3D implementation. This return was not simply a reversal, but the result of iterative refinement, where lessons from the simplified 2D stage informed a more grounded and purposeful 3D design.

UX and UI considerations were especially important in this methodology because the project is centred on the design of an interactive experience rather than a purely technical system. Beginner accessibility was a key goal, meaning that the interface needed to feel intuitive, clear, and easy to learn without prior knowledge of digital audio workstations or synthesis terminology. At the same time, the system also needed to remain meaningful for experienced producers, who would already be familiar with conventional controls such as sliders and knobs. For this reason, the interface was designed not as a replacement for professional tools, but as an alternative workflow that could still support serious creative exploration. Immediate audio-visual feedback was also essential, since users needed to understand the effect of their actions as they manipulated the shape in real time. These considerations influenced the simplification of the interface, the clarity of the visual language, and the design of the mapping model itself.

Testing also took place informally throughout development, rather than being confined to a single final stage. Both beginner users and experienced producers were asked to use the application during different stages of development, and their interactions were observed in order to identify weaknesses in the UI and UX. This allowed practical issues to be addressed as they emerged, making testing part of the iterative design process rather than something applied only after implementation. For example, one user suggested the ability to lock notes into a key so that melodies could be created more easily without drifting out of tune. This feature was later implemented, as it strongly supports the ideation stage of music production and makes the system more approachable for users who may not have strong music theory knowledge. In this way, informal user feedback directly influenced refinements to both the functionality and the usability of the project.

This testing approach is suitable for the project because The Shape of Sound is intended to serve two distinct but related user groups: beginners who need an accessible and unintimidating introduction to sound design, and experienced producers who may value the tool as a fresh creative workflow. Testing with beginner users helps evaluate whether the interface is intuitive, whether the visual mappings make sense without prior technical knowledge, and whether the system reduces the intimidation often associated with traditional music software. Testing with more experienced producers is equally important, as it helps assess whether the tool offers genuine creative value rather than functioning only as a simplified educational exercise. Using both groups therefore reflects the dual aims of the project, allowing the methodology to evaluate not only usability and learnability, but also the broader creative usefulness of the tool.

Overall, this methodology was well suited to the aims of the project because it allowed technical development, interaction design, and user feedback to inform one another throughout the process. The staged plan provided structure and clear milestones, while iterative prototyping made it possible to refine the tool in response to practical findings as development progressed. This was particularly important for a project whose success depended not only on technical implementation, but on whether the interface felt intuitive, expressive, and educational in use. By combining structured planning with iterative design and user-centred evaluation, the methodology supported the development of a tool that could be shaped gradually in relation to both its creative goals and its intended users.

unfinished: this section still needs a more formal testing and evaluation paragraph once the structured testing stage has been completed. also not quite long enough

5 Design

The design of The Shape of Sound was centred on creating a visual-first web application that could make sound design more intuitive without removing creative depth. The final design needed to support two related aims: accessibility for beginners who may find conventional music software intimidating, and creative usefulness for more experienced producers seeking an alternative workflow for experimentation. For this reason, the system was designed not only as a technical implementation of audio synthesis, but as a carefully structured interaction model in which visual form, interface layout, and sound behaviour work together. This chapter focuses on the final software structure, the conceptual design of the interaction model, the organisation of the interface, and the supporting flows that allow the application to function as a complete product.

At a high level, the final architecture is organised into four connected layers. The first is the application layer, where Next.js structures the web application and its routes, while React manages the surrounding interface and shared state. The second is the visual interaction layer, where Three.js renders and updates the shape-based interface that users manipulate directly. The third is the audio layer, where Tone.js drives synthesis behaviour and responds to changes in the visual state. The fourth is the supporting product layer, where Better Auth, PostgreSQL, Zod schemas, and wasm-media-encoders support accounts, persistence, validation, and export. Although these supporting systems are important to the delivered product, the core of the design remains the synchronisation between the visual layer and the audio layer, since this relationship defines how the application is experienced by the user.

The central interaction model was designed as a synesthetic interface in which users shape sound by manipulating visual form. Rather than relying only on conventional synthesis controls such as parameter sliders, the design supplements those ideas with a more intuitive layer built around shape, colour, size, roundness, wobble, and pitch-related control. These properties were chosen because they can be understood visually and can be mapped to audible change in ways that feel immediate and expressive, even

for users with little prior experience of synthesis. This was especially important for encouraging stronger creative exploration, while also supporting learning and emotional engagement through direct interaction. Features such as key lock further support this model by reducing friction during experimentation, allowing users to explore ideas freely without needing advanced music theory knowledge. Immediate audio feedback was essential to this design, as it allowed each visual change to be understood through hearing as well as sight, reinforcing the relationship between interaction and sound in real time.

Control / property	Mapped audio behaviour	Design purpose
Shape preset	Selects the base oscillator waveform	Gives users an immediate way to choose between different sound characters.
Colour	Selects note / pitch class	Links pitch choice to a visual identity that can be read quickly.
Octave	Shifts pitch register	Allows melodic range to change without altering the main interaction model.
Key / scale lock	Constrains note choice to a selected key and scale	Reduces friction during melodic experimentation and supports beginner accessibility.
Size	Controls output gain / perceived loudness	Makes changes in intensity visually legible.
Roundness	Crossfades toward a smoother sine-based timbre	Connects visual softness with sonic smoothness.
Wobble	Controls tremolo depth	Turns visible movement into audible modulation.
Wobble speed	Controls tremolo rate	Lets motion intensity vary over time rather than remaining static.
Envelope controls	Shape attack, hold, decay, and sustain behaviour	Gives users control over the temporal character of the sound.

Table 1: Core interaction mappings used in The Shape of Sound.

The UI and layout design were organised so that the interactive shape remained the clear focal point of the application. The main visual interface was treated as the centre of the experience, while the surrounding controls were arranged to support sound shaping without overwhelming the user or distracting from the primary interaction. This relationship between the shape view and the control interface was important because the project needed to remain visually clear, easy to approach, and quick to understand, especially for beginners encountering synthesis concepts for the first time. At the same time, the layout also needed to remain useful for experienced producers by allowing fast experimentation, note input, key-lock interaction, and access to supporting functions such as export, settings, and saved work. In this way, the interface design aimed to create a clear visual hierarchy in which the shape remained primary, key controls remained immediately accessible, and secondary product features supported the workflow without competing with the core audiovisual interaction.

The final design was also shaped by a clear process of refinement across multiple stages of development. The initial direction emphasised a 3D interface, as this best reflected the ambition of creating a visually expressive and sculptural interaction model for sound design. During development, however, it became useful to move temporarily into a simpler 2D Konva-based stage, where the project could test interaction clarity, mapping behaviour, and usability without the added complexity of full 3D scene management. This reduced phase was especially valuable for identifying which parts of the design were essential to the user experience and which were only visually ambitious. Once these interaction principles became clearer, the project returned to a final 3D implementation, but with a more grounded sense of purpose. As a result, the final design can be understood not as the product of a single uninterrupted idea, but as an iterative refinement process in which each stage informed the next.

The internal structure of the system was designed around a small number of shared state and data models so that visual behaviour, audio behaviour, and persistence could be consistent. At the centre of the editor is a single shape state object that holds the current sound-design values, including preset, colour, roundness, size, wobble, envelope settings, octave, and scale-related information, while editor-specific values such as rotation and position are handled alongside it. This state is managed centrally and then passed into the main interface components, allowing the visual canvas, control panels, keyboard input, preview behaviour, and export features to remain synchronised. The component structure reflects this organisation: a shared layout frames the application, canvas components handle visual rendering and manipulation, and control components handle parameter input, settings, and supporting actions. Persistence is built on a related but cleaner sound-configuration model, which allows ideas to be saved in a database as user-owned records with titles, timestamps, and validated sound settings, while also supporting local draft storage for unauthenticated use. This separation between interactive editor state, reusable sound configuration, and persisted idea records helps the final design function as both an instrument and a complete web product.

The functional design of the system was organised around a small number of key user flows that reflect the main purpose of the application. The most important of these is the creation of a sound through direct manipulation of the visual shape, where changes to properties such as colour, roundness, size, wobble, octave, and scale settings produce immediate sonic feedback. This core flow is supported by note input and key-lock functionality, allowing users to move from exploratory sound shaping into the creation of short melodic ideas without leaving the main interaction space. Beyond the audiovisual core, the design also includes supporting product flows such as account creation, saving and loading ideas, local draft persistence for unauthenticated use, and export of completed sounds for use outside the application. These flows were designed to remain secondary to the main interaction, so that they extend the usefulness of the tool without disrupting its identity as a visual-first sound design environment.

At the level of interaction technique, the design depends on a set of mappings and transformations that make the system feel coherent as an instrument rather than as a collection of disconnected controls. Visual-to-audio parameter mapping is central to this, since the design only succeeds if changes in shape are translated into sound in ways that feel understandable and expressive. Roundness and waveform behaviour are linked through blending and crossfading, allowing visual smoothness to correspond with smoother sonic character. Wobble introduces a further layer of interaction by turning visible movement into audible modulation, so that the shape feels active rather than static. Shape interpolation and morphing support gradual transformation rather than abrupt switching, helping the visual and sonic behaviour remain continuous. Scale and key-lock design further reduce friction by allowing users to explore melodies more freely without immediately encountering harmonic errors. Taken together, these design decisions ensure that the interaction model remains learnable for beginners while still providing enough depth to support more exploratory and musically meaningful use.

Taken as a whole, the final design addresses the main requirements of the project by combining accessibility, immediate feedback, creative usefulness, and practical workflow support within a single system. The visual-first interaction model responds directly to the goal of reducing intimidation for beginners, while the depth of the mapping and sound-shaping system helps preserve value for more experienced users. Supporting features such as persistence, account functionality, and export extend the design beyond a self-contained experiment, allowing the application to function as a complete product rather than only a prototype. In this way, the design of The Shape of Sound is not only a response to technical needs, but also a direct response to the educational, creative, and experiential aims established earlier in the project.

6 Tools and Implementation

6.1 Tools used and justification

The implementation of The Shape of Sound required a stack that could support a visual-first interface, real-time browser-based sound synthesis, and the wider infrastructure of a complete web product. For this reason, the tools used in the project were chosen not

only for technical capability, but also for how well they supported accessibility, creative exploration, persistence, and export.

The frontend application was built with Next.js, React 19, and TypeScript. Next.js was suitable because the project needed more than a self-contained interaction demo: it also required routing, API endpoints, account-related functionality, persistence flows, and a clear application structure around the audiovisual core [28]. React supported the project through its component model and state management patterns, making it easier to organise interactive controls, shared state, and the overall layout of the interface [30]. TypeScript added stronger type safety across shape state, audio mappings, persistence models, and API data, helping the system remain maintainable as its complexity increased [51]. This part of the stack also aligned well with my existing experience in React-based web development, which reduced the need for unnecessary framework learning.

For visual interaction, the final system uses Three.js to render the main shape interface in 3D. This was an important choice because the project depends on visual form being central to the interaction model, and a 3D environment allows the shape to feel more spatial and expressive than a flatter 2D presentation [32]. During development, a 2D Konva-based phase was also explored as a way to simplify the interface and test interaction ideas more quickly [33]. That stage was useful for clarifying usability and mapping behaviour, but the final return to Three.js made it possible to recover the visual richness that better matched the intended identity of the project.

For audio, the project uses Tone.js alongside the native Web Audio API. Tone.js was a practical choice because it provides higher-level abstractions for synthesis, timing, and musical behaviour, making it possible to build and test sound features more quickly than would have been practical with raw browser audio code alone [34]. This was especially helpful because the main challenge of the project was not building a synthesis engine from first principles, but designing meaningful mappings between visual interaction and sound. At the same time, the Web Audio API remains the technical foundation of the audio system, since Tone.js still depends on browser-native audio infrastructure [52].

Several supporting tools were also used to turn the project into a complete product rather than only an interaction prototype. Tailwind CSS supported rapid and consistent interface styling [36]. Better Auth handled authentication and session management, while Drizzle ORM, PostgreSQL, and Neon provided the persistence layer for saved ideas and user data [37], [38], [39]. Zod was used to validate schemas and request payloads [53], and wasm-media-encoders made browser-generated sounds portable beyond the live session through export [40]. Although these tools were not the centre of the project's creative contribution, they were justified because they supported the final product scope and improved the reliability and usability of the system.

This part of the project involved a mixture of existing skills and new learning. Prior experience with React-based web development, TypeScript, and general full-stack application structure made it possible to work confidently with the broader application

framework. However, the project also required deeper development in browser-based audio, visual interaction design, shape-to-sound mapping, and browser-side export. This combination of familiar tools and new technical challenges was appropriate for the project, as it kept the implementation feasible while still supporting genuine skill development.

6.2 Implementation

This section explains how the final system was implemented through the main use cases of the application, rather than describing the codebase file by file. The focus is on how the core interaction model, persistence features, and export workflow were translated into working code, and how these implementation decisions reflect the design choices discussed in the previous chapter.

The most important use case is the creation and shaping of a sound through direct interaction with the visual editor. This behaviour is centred on a shared shape state model that stores the current sound-design values, including preset, colour, roundness, size, wobble, envelope settings, octave, and scale information. In practice, this means that the same state can drive both the visible behaviour of the shape and the audible behaviour of the synthesizer, keeping the interaction model coherent. When a user changes a control, such as the preset selector, colour keyboard, or roundness slider, the editor updates the shared state and then either triggers or prepares a preview sound using the current mapped values. The canvas, control panel, and preview audio are therefore synchronised through the same state rather than treated as separate systems. This is important because the implementation needs to preserve the central design goal of immediate audio-visual feedback: a change in the interface must feel like a change in the instrument itself, not a delayed or disconnected update.

At the audio level, these state values are not used directly. Instead, they are translated through a set of mapping functions into synthesizer behaviour that Tone.js can apply consistently across preview and export. The selected preset determines the base oscillator type, roundness controls the crossfade toward a smoother sine-based tone, and size is converted into gain so that visual scale affects perceived loudness. Envelope values are mapped into attack, hold, decay, and sustain timings, while wobble and wobble speed become tremolo depth and rate. In this way, the implementation keeps a clear separation between user-facing controls and low-level sound behaviour: users interact with meaningful visual properties, while the synthesizer receives the mapped values needed to produce coherent audio output.

In practice, preview playback is built around a small collection of audio nodes that are assembled each time a note or sample is played. The implementation creates a primary oscillator, a secondary sine oscillator, a crossfade node, and gain stages for envelope shaping and tremolo behaviour. This structure is important because it mirrors the design of the interaction model: the shape preset selects the main oscillator character, and roundness blends it toward a smoother sine tone. The resulting signal then passes through gain stages that apply the envelope and modulation behaviour before reaching

the output. Each part of the signal chain therefore reflects a clear aspect of the user-facing design, and the same implementation pattern can be reused when previewing notes, holding sustained playback, or rendering exports.

A simplified version of the central state model is shown below:

```
type ShapeState = {
  preset: "triangle" | "square"
  color: string
  roundness: number
  size: number
  wobble: number
  wobbleSpeed: number
  attack: number
  hold: number
  decay: number
  sustain: number
  octave: number
  keyRoot: NoteName | null
  scaleType: ScaleType
  x: number
  y: number
}
```

Another important use case is note input and scale-aware interaction. The editor supports direct note entry through both the colour keyboard and mapped computer-keyboard controls, allowing users to move beyond single note exploration into short melodic ideas. In this part of the implementation, colour is not only a visual choice but also a pitch selection system, since each colour corresponds to a note. The key-lock and scale-selection features extend this by remapping note input so that users can stay within a chosen major or minor scale. This reduces friction during composition, particularly for beginners who may not have strong music theory knowledge, while still remaining useful for experienced producers during rapid ideation. In implementation terms, this behaviour is handled through scale-note generation, note-in-scale checks, and diatonic remapping of keyboard input, allowing the interface to preserve musical constraints without interrupting the nature of the interaction.

```
if key lock is off:
  play the selected note and octave directly
else:
  generate notes in the selected scale
  remap keyboard input to the nearest scale note position
  play the remapped note within the current octave range
```

Persistence is implemented through two related flows: local drafts for unauthenticated users and database-backed Idea records for authenticated users. While exploring the editor on the root page, users who are not signed in can still have their current sound preserved through a lightweight local draft system stored in the browser. Once authenticated, users can create, update, load, and delete saved ideas through API routes backed by validated request schemas and database persistence. This implementation is

important because it allows the product to remain approachable for casual experimentation while still supporting longer-term ownership and reuse of work. In practice, the sound editor state is converted into a reusable sound-configuration object, validated with Zod, and then either stored locally or sent through the ideas API for persistence in PostgreSQL. This means the same core sound representation can move across guest use, authenticated saving, public viewing, and later editing without needing separate implementations for each mode.

In implementation terms, this flow is split deliberately between client-side convenience and server-side persistence. Local drafts are stored through a small browser storage layer, where the current sound configuration is converted into a lightweight draft record, validated against a schema, sorted by save time, and limited to the most recent draft. This keeps casual use simple while preventing local draft storage from expanding unnecessarily. For authenticated users, persistence moves through the API layer, where requests are validated with Zod before the database is touched, and ownership is checked before records can be updated or deleted. The server exposes separate routes for listing ideas, creating new records, reading a public or owned record, updating existing ideas, deleting them, and importing local drafts after sign-in. This separation is valuable because it means guest experimentation, authenticated ownership, and public sharing can all be supported through the same underlying sound configuration model, while still preserving clear boundaries between temporary local state and durable saved data.

A simplified version of the persisted Idea schema is shown below:

```
export const idea = pgTable("ideas", {
  id: uuid("id").primaryKey(),
  ownerUserId: text("owner_user_id").notNull(),
  title: varchar("title", { length: 120 }).notNull(),
  config: jsonb("config").$type<IdeaSoundConfig>().notNull(),
  createdAt: timestamp("created_at", { withTimezone: true, mode:
    "date" }).notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true, mode:
    "date" }).notNull(),
})
```

The final user-facing flow is export, which allows the generated sound to move beyond live interaction in the browser and into wider music-making workflows. This was implemented through an export dialog that lets the user choose format and whether the current envelope settings should be applied. Under the hood, the current state is rendered offline into an audio buffer rather than being recorded from live playback, which produces a cleaner and more controlled export path. The resulting buffer is then encoded as WAV or MP3 and downloaded directly in the browser. This implementation is important because it turns the application from a purely exploratory interface into a practical creative tool, allowing users to keep and reuse the sounds they create. In this way, export is not only a convenience feature, but part of how the application supports real ideation and integration with broader production workflows.

The choice to render offline rather than record the live output was important because it keeps the export pipeline deterministic and independent of playback conditions in the browser. Instead of capturing whatever happens to be playing in real time, the implementation reconstructs the current sound state in an offline render, applies the selected note, waveform blend, gain, and optional envelope, and then generates a fresh audio buffer specifically for export. This makes the result more predictable and avoids issues that could arise from live preview behaviour, timing differences, or muted output. The export dialog also exposes practical options that reflect different producer workflows: WAV is useful as an uncompressed format, MP3 offers a smaller portable file, and the option to apply or omit the current envelope lets users choose between a shaped sound or a drier sample that can be processed later in another environment. This means the export system is closely tied to the project's goal of supporting creativity beyond the browser session, not just technically producing a file.

Across these use cases, several shared implementation techniques appear repeatedly. Visual-to-audio mapping functions translate editor values such as preset, roundness, size, and wobble into synthesis behaviour, allowing the code to keep the interaction model consistent across preview, playback, and export. Crossfading and parameter automation are used so that visual changes can produce smooth sonic transitions rather than abrupt jumps, while tremolo-style modulation makes visible movement audibly active. Validation also plays an important role, as Zod schemas are used to keep saved idea payloads consistent before they are written to persistence layers or exposed through API responses. Where requests modify stored data, server-side rate limiting is also applied so that persistence features remain more robust and less open to abuse. These shared techniques are important because they allow the system to behave consistently across different features, while keeping the core experience of the instrument coherent for the user.

One important implementation decision was to isolate the mapping logic into a dedicated layer rather than embedding it directly into UI components or synthesis setup code. Functions such as preset-to-waveform mapping, roundness-to-crossfade mapping, size-to-gain mapping, and wobble-to-tremolo mapping make it possible for the same interaction rules to be reused whenever sound is previewed, played, or exported. This helps preserve consistency across the system and makes the code easier to reason about, because the visual editor does not need to know the low-level audio values required by the synthesizer. In a similar way, crossfading and parameter automation are used to preserve continuity between visual and sonic change, so that smooth changes in the shape correspond to smooth changes in timbre or amplitude rather than abrupt jumps.

Validation and request protection form another shared layer of implementation. Zod schemas are used to describe the expected shape of saved sound configurations, local draft payloads, and API requests, which means invalid data can be rejected before it reaches persistence or public responses. This is especially important in a system where the same sound configuration must move across browser state, local drafts, authenticated records, and exports. Server-side rate limiting is applied to write-heavy

routes such as create, update, delete, and local-draft import, which helps keep these features more robust in practice. Although these techniques are less visible than the audiovisual interaction itself, they are essential in making the final application reliable and coherent as a product rather than only as a prototype.

Not all parts of the implementation were invented from scratch, and it is important to distinguish custom project logic from standard technical patterns. The application uses established approaches taken from official documentation and common library usage for areas such as Next.js route handlers, Drizzle schema design, browser audio handling, and export workflows. In these cases, the code follows recognised implementation patterns because they are reliable and appropriate to the tools being used. The more original contribution of the project lies in how these established technologies were combined with a custom interaction model, especially in the shape-to-sound mappings, the synchronisation of visual and audio behaviour, and the design of a visual-first synthesis workflow.

Taken together, the implementation reflects the main aims of the project by turning the design into a system that is interactive, understandable, and practically usable. The code does not only support real-time audiovisual behaviour, but also persistence, export, and multiple modes of use across guest and authenticated interaction. Organising the implementation by use case makes it clear that the application was built around what users need to do with the system, rather than around isolated technical components. In this way, the implementation supports the project's broader goals of accessibility, creative exploration, and usable browser-based sound design.

7 Testing

7.1 Functional testing

Functional testing was used to evaluate whether the final application met its main requirements and behaved correctly from the perspective of the user. Because The Shape of Sound is an interactive web application, this testing focused primarily on black-box behaviour: whether core actions produced the expected outputs when used through the interface. The purpose of this stage was to confirm that the implemented features worked reliably across the main user-facing flows, including sound creation, parameter control, key and scale interaction, persistence, export, and access behaviour.

The functional test cases were derived directly from the core requirements established earlier in the project, which allowed the testing process to remain clearly linked to the intended behaviour of the final system. In practice, this meant testing the application through the same actions a user would perform, rather than testing isolated internal functions in abstraction. The emphasis was therefore on requirement coverage and observable behaviour, with each test case checking whether a complete feature or flow performed as expected. This approach was appropriate because the success of the application depends not only on internal correctness, but on whether the implemented features behave consistently and reliably when used through the interface.

Although the testing strategy was mainly black-box in nature, some white-box awareness still informed the choice of test cases. The application contains distinct technical areas such as audiovisual state mapping, authenticated persistence, local draft storage, schema validation, and browser-side export, and knowledge of these internal responsibilities helped determine which user-facing behaviours needed explicit verification. For example, persistence was not treated as a single feature, but as a set of related flows involving browser storage, API routes, validation, ownership checks, and database-backed records. In the same way, export testing needed to cover both the user-facing options in the dialog and the successful production of downloadable output files. This balance kept the tests grounded in real use while still ensuring that implementation-sensitive behaviour was not overlooked.

The tests were carried out in the browser environment in which the application is actually intended to be used. This was important because the system depends on the interaction between UI controls, shared state, browser audio behaviour, authentication state, and file generation rather than on isolated computational output alone. Test cases were therefore performed through the live interface, with both signed-in and signed-out conditions used where relevant. This allowed persistence, guest draft behaviour, public access, and export to be checked in the same environment in which real users would encounter them. Using the running application in this way helped ensure that the results reflected practical system behaviour rather than only isolated code-level correctness.

Test	Requirement	Test case	Expected result	Result
FT-01	Core sound creation	Change shape preset and confirm the sound character updates.	Preset change updates the mapped waveform behaviour.	✓
FT-02	Visual parameter controls	Adjust size, roundness, and wobble controls during playback.	Audio output responds immediately to each control change.	✓
FT-03	Pitch interaction	Select different colours/ notes and octave values.	Pitch changes correctly according to note and octave.	✓
FT-04	Key lock / scale	Enable key lock and play notes from the keyboard.	Input is constrained to the selected key and scale.	✓
FT-05	Persistence	Save an idea while authenticated, then reload it.	Saved configuration is restored correctly.	✓
FT-06	Guest draft support	Use the editor while signed out and revisit the draft state.	Local draft remains available for continued use.	✓
FT-07	Export	Export the current sound in WAV and MP3 formats.	Audio file downloads successfully in the selected format.	✓
FT-08	Public access / sharing	Open a saved public idea by URL while not signed in.	Idea loads for playback while editing remains restricted.	✓

Table 2: Functional test plan and outcomes for the final application.

The results shown in Table 2 indicate that the main functional requirements of the application were met in the final version of the system. Core sound creation, parameter manipulation, note and scale behaviour, persistence, export, and public access flows all performed as expected when tested through the interface. These results provide evidence that the implemented features behave reliably and that the system is technically capable of supporting its intended use cases. However, it is important to note that functional testing alone does not measure the usability or creative quality of the application. While the features may work correctly from a technical standpoint, this does not necessarily mean that they provide a satisfying or effective user experience. Functional testing was therefore an important step in confirming that later user evaluation could focus on experience and usefulness rather than on unresolved implementation failures.

Functional testing was also carried out iteratively rather than only once at the end of development. As features such as key-lock behaviour, save and load flows, export options, and public idea access were added or refined, the relevant tests were repeated to check that new work had not broken previously working behaviour. This form of regression checking was especially important in a browser-based system where UI changes, state updates, persistence logic, and audio behaviour are closely connected. Running these checks through the live interface also helped confirm that the application behaved correctly in the same environment in which it was intended to be used. In this sense, the functional testing process did not only produce a final set of pass results, but also supported stability throughout development.

7.2 User testing

User testing was used to evaluate how the final output performed as an interactive creative tool, rather than only as a technically correct application. This was important because the aims of The Shape of Sound go beyond its feature-set: the project is also concerned with whether sound design feels approachable, intuitive, and creatively useful when experienced through a visual interface. For this reason, user testing focused on usability, perceived intuitiveness, approachability, and creative value.

The evaluation involved five participants, consisting of three beginners and two experienced producers. This split was appropriate because the project was designed for two main user groups: users with little or no prior experience of music production, and users who already work with conventional production tools. Beginner participants were important for testing accessibility and first-use clarity, while experienced producers were important for judging whether the application offered meaningful value as a creative workflow rather than only as a simplified educational tool.

Participants were asked to complete a small set of tasks using the live application, including creating a sound, changing the shape and describing the resulting audio change, making a short melody, using the key-lock feature, and exporting the result. After completing these tasks, they filled in a short questionnaire using a five-point scale and written feedback prompts. This approach made it possible to collect both structured results and more reflective comments. The summary below includes the completed responses gathered during this stage of evaluation.

Person	Experience	Easy	Intuit.	Approach.	Key	Useful	Reusability
A	Beginner	5	4	5	5	5	5
B	Beginner	4	4	5	5	4	4
C	Experienced	4	5	4	4	5	5
D	Beginner	3	4	4	5	4	4
E	Experienced	4	4	4	4	5	5
AVG	Mixed	4.0	4.2	4.4	4.6	4.6	4.6

Table 3: Summary of user testing questionnaire ratings.

Person	Key feedback
A	Dragging the control points was not always clear, and the keyboard view could better show which notes remain available when key lock is active.
B	The controls became clearer after experimentation, and the key lock helped make something musical quickly.
C	The shape interface was useful for exploring sound ideas, but some parameter ranges could be clearer.
D	The first-use flow was less obvious, but experimentation became easier after understanding the main controls.
E	The tool felt strongest as a creative sketchpad, though the visual mapping to familiar synth ideas could be clearer.

Table 4: Summary of key feedback from user testing.

The ratings shown in Table 3 suggest that the application performed well across both beginner and experienced participants. The average scores are consistently high, with approachability, creative usefulness, and reusability all scoring above 4.0, and key-lock receiving the strongest average rating at 4.6. This is significant because it suggests that the system was not only understandable, but also practically useful during melodic experimentation. The slightly lower average for ease of understanding indicates that the interface still requires some onboarding, but the overall pattern suggests that participants were generally able to learn the system quickly and derive value from it. The results also show that the tool was appreciated by both intended user groups, rather than appealing only to beginners or only to experienced producers.

The feedback in Table 4 provides a more detailed view of where the design is already successful and where it still needs refinement. Positive comments consistently focused on the immediacy of the interaction, the ease of experimenting with sound, and the usefulness of the application as a sketchpad for generating ideas. This aligns well with the project's aim of supporting creative exploration through a visual-first workflow. At the same time, several responses pointed to issues of clarity, particularly around control-point interaction, keyboard visibility during key-lock use, first-use guidance, and the visibility of parameter ranges. These comments suggest that the main remaining weaknesses are not in the core concept itself, but in how clearly the interface communicates that concept to the user. This is valuable because it directs further refinement toward onboarding, clarity, and visual explanation rather than requiring a redesign of the core interaction model.

Taken together, the user testing indicates that The Shape of Sound is already functioning effectively as both an accessible introduction to sound design and a creatively useful experimental tool. The strongest areas appear to be approachability, melodic support through key lock, and the sense of creative usefulness reported by participants. The weaker areas are more closely related to first-use communication and control clarity. As a result, the findings from this stage of testing support the overall direction of the project while also identifying concrete areas for improvement in later refinement.

8 Conclusions and Reflections

Bibliography

- [1] EDMProd, “What's the Biggest Struggle in Music Production? A Survey.” [Online]. Available: <https://www.edmprod.com/music-production-struggles/>
- [2] N. K. Nair and S. Chatterjee, “Musician's Block in Contemporary Music Artists: Coping Mechanisms in the Contemporary Era,” *International Journal For Multidisciplinary Research*, 2025, [Online]. Available: <https://www.ijfmr.com/papers/2025/2/42148.pdf>
- [3] Karanyi Sounds, “How to Overcome Creative Block in Music Production: Traditional and Innovative Solutions.” [Online]. Available: <https://karanyisounds.com/blogs/production-tips/how-to-overcome-creative-block-in-music-production-traditional-amp-innovative-solutions>
- [4] M. Gladwell, *Outliers: The Story of Success*. 2008.
- [5] L. Shams and A. R. Seitz, “Benefits of multisensory learning,” *Trends in Cognitive Sciences*, vol. 12, no. 11, pp. 411–417, 2008.
- [6] R. Sigrist, G. Rauter, R. Riener, and P. Wolf, “Augmented visual, auditory, haptic, and multimodal feedback in motor learning: A review,” *Psychonomic Bulletin & Review*, vol. 20, no. 1, pp. 21–53, 2013.
- [7] C. Spence, “Crossmodal correspondences: A tutorial review,” *Attention, Perception, & Psychophysics*, vol. 73, pp. 971–995, 2011.
- [8] D. M. Sidhu and P. M. Pexman, “Five mechanisms of sound symbolic association,” *Psychonomic Bulletin & Review*, vol. 25, no. 5, pp. 1619–1643, 2018.
- [9] A. Cwiek, S. Fuchs, C. Draxler, and others, “The bouba/kiki effect is robust across cultures and writing systems,” *Philosophical Transactions of the Royal Society B*, vol. 377, p. 20200390, 2022.
- [10] Wikipedia, “Clavier à lumières.” [Online]. Available: https://en.wikipedia.org/wiki/Clavier_%C3%A0_lumi%C3%A8res
- [11] N. Schwartz, A. Snir, and A. Amedi, “Feeling the music: exploring emotional effects of auditory-tactile musical experiences,” *Frontiers in Virtual Reality*, 2025.
- [12] T. Hermann, A. Hunt, and J. G. Neuhoff, Eds., *The Sonification Handbook*. 2011.
- [13] D. Rocchesso and S. Serafin, “Sonic Interaction Design,” *International Journal of Human-Computer Studies*, vol. 67, no. 11, pp. 905–906, 2009.
- [14] A. P. McPherson, F. Morreale, and J. Harrison, “Musical Instruments for Novices: Comparing NIME, HCI and Crowdfunding Approaches,” *New Directions in Music and Human-Computer Interaction*. pp. 179–212, 2019.

- [15] B. Shneiderman, “Creativity support tools: Accelerating discovery and innovation,” *Communications of the ACM*, vol. 50, no. 12, pp. 20–32, 2007.
- [16] A. Tsiros and G. Leplatre, “Utilising Natural Cross-Modal Mappings for Visual Control of Feature-Based Sound Synthesis,” in *Proceedings of the 19th ACM International Conference on Multimodal Interaction*, 2017.
- [17] Z. Sramek, A. J. Sato, Z. Zhou, S. Hosio, and K. Yatani, “SoundTraveller: Exploring Abstraction and Entanglement in Timbre Creation Interfaces for Synthesizers,” in *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, 2023, pp. 95–114.
- [18] Image-Line, “Create Your Best Music | FL Studio.” [Online]. Available: <https://www.image-line.com/>
- [19] Image-Line, “Piano roll.” [Online]. Available: <https://www.image-line.com/fl-studio-learning/fl-studio-online-manual/html/pianoroll.htm>
- [20] Ableton, “Welcome to Live — Ableton Reference Manual Version 12.” [Online]. Available: <https://www.ableton.com/en/manual/welcome-to-live/>
- [21] Ableton, “Accessibility Options in Live.” [Online]. Available: <https://www.ableton.com/en/live-manual/12/accessibility-options-in-live/>
- [22] So Far So Good, “Incredibox.” [Online]. Available: <https://www.incredibox.com/>
- [23] So Far So Good, “Incredibox for schools.” [Online]. Available: <https://www.incredibox.com/schools>
- [24] Splice, “Beatmaker: Make a Beat for Free.” [Online]. Available: <https://splice.com/sounds/beatmaker>
- [25] Splice, “Introducing Beat Maker for Splice Sounds.” [Online]. Available: <https://splice.com/blog/introducing-beat-maker-for-splice-sounds/>
- [26] Synesthesia, “Synesthesia - Live Music Visualizer - VJ Software.” [Online]. Available: <https://synesthesia.live/>
- [27] Synesthesia, “Features.” [Online]. Available: <https://synesthesia.live/features>
- [28] Vercel, “Next.js Docs.” [Online]. Available: <https://nextjs.org/docs>
- [29] Vercel, “Next.js 15.” [Online]. Available: <https://nextjs.org/blog/next-15>
- [30] React Team, “React 19.” [Online]. Available: <https://react.dev/blog/2024/12/05/react-19>
- [31] three.js authors, “three.js.” [Online]. Available: <https://threejs.org/>
- [32] three.js authors, “three.js Manual.” [Online]. Available: <https://threejs.org/manual/>
- [33] Konva contributors, “Getting Started with Konva.” [Online]. Available: <https://konvajs.org/docs/index.html>

- [34] Tone.js contributors, “Tone.js Documentation.” [Online]. Available: <https://tonejs.github.io/docs/>
- [35] Tone.js contributors, “Tone.js.” [Online]. Available: <https://tonejs.github.io/>
- [36] Tailwind Labs, “Styling with utility classes.” [Online]. Available: <https://tailwindcss.com/docs/styling-with-utility-classes>
- [37] Better Auth, “Introduction | Better Auth.” [Online]. Available: <https://www.better-auth.com/docs>
- [38] Drizzle Team, “Why Drizzle?.” [Online]. Available: <https://orm.drizzle.team/docs/overview>
- [39] Neon, “Neon documentation - Introduction.” [Online]. Available: <https://neon.com/docs/introduction>
- [40] arseneyr, “wasm-media-encoders.” [Online]. Available: <https://github.com/arseneyr/wasm-media-encoders>
- [41] A. Hunt, M. M. Wanderley, and M. Paradis, “The Importance of Parameter Mapping in Electronic Instrument Design,” in *Proceedings of the 2002 Conference on New Interfaces for Musical Expression*, 2002. [Online]. Available: https://www.nime.org/proceedings/2002/nime2002_088.pdf
- [42] A. Tanaka and A. Parkinson, “On mapping as a technoscientific practice in digital musical instruments,” *Journal of New Music Research*, 2025, [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/09298215.2024.2442356?src=exp-la>
- [43] MDN Web Docs, “AudioParam.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/AudioParam>
- [44] MDN Web Docs, “AudioParam: linearRampToValueAtTime() method.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/AudioParam/linearRampToValueAtTime>
- [45] Tone.js contributors, “Tremolo.” [Online]. Available: <https://tonejs.github.io/docs/15.0.4/classes/Tremolo.html>
- [46] MDN Web Docs, “GainNode.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/GainNode>
- [47] M. Alexa, D. Cohen-Or, and D. Levin, “As-Rigid-As-Possible Shape Interpolation,” in *Proceedings of SIGGRAPH 2000*, 2000. [Online]. Available: <https://dl.acm.org/doi/10.1145/344779.344859>
- [48] G. Wolberg, “Image morphing: a survey,” *The Visual Computer*, vol. 14, no. 8–9, pp. 360–372, 1998, [Online]. Available: <https://pages.cs.wisc.edu/~lizhang/courses/cs559-2010s/syllabus/02-10-warping/Wolberg1998IMS.pdf>

- [49] MDN Web Docs, “MediaStream Recording API.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaStream_Recording_API
- [50] MDN Web Docs, “AudioContext: createMediaStreamDestination() method.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/AudioContext/createMediaStreamDestination>
- [51] Microsoft, “TypeScript Handbook.” [Online]. Available: <https://www.typescriptlang.org/docs>
- [52] World Wide Web Consortium, “Web Audio API 1.1.” [Online]. Available: <https://www.w3.org/TR/webaudio-1.1/>
- [53] Zod Contributors, “Zod Documentation.” [Online]. Available: <https://zod.dev/>